

WEEK 3 · PROJECT DOCUMENTATION

VirtualSoccerArena

soccer-arena-mcp — an agentic indoor-arena booking system

Program	Mastering Agentic AI Bootcamp · The Gen Academy · Week 3
Author	Lawin Sunny Bunga
Track	Bring-your-own use case · Code track (custom MCP tools, with Claude Desktop as the agent runtime)
Repository	soccer-arena-mcp (GitHub)

Contents

1 The One-Liner (Primer)

My agent helps a soccer player book and manage indoor-arena sessions in Claude Desktop, replacing the 15–20 minutes of checking a website for open slots, reading policy PDFs, and filling out a booking form. It checks availability, answers policy questions, checks the weather, and books a slot on its own using **4 MCP tools**, hands off to the human to confirm before any booking is created or emailed, and I'll know it works when a user can go from “is anything open Saturday?” to a confirmed booking with a code in under 2 minutes, correctly about 9 times out of 10.

2 The Agent Framework

Field	Detail
Agent goal	Book and manage 2-hour indoor soccer-arena sessions through natural conversation.
Where it's used	Claude Desktop (local stdio MCP server), with a Next.js companion web app (Toronto-themed, CN Tower skyline) for viewing info and a code-locked mini-game.
Steps, in order	(1) Interpret the request → (2) call read tools as needed (<code>check_availability</code> , <code>query_policy</code> , <code>get_weather</code>) → (3) confirm date / slot / name / email with the user → (4) call the write tool <code>create_booking</code> → (5) return the booking code and trigger the emailed .ics.
What it can do (tools)	<code>check_availability</code> (read), <code>query_policy</code> (read — RAG), <code>get_weather</code> (read), <code>create_booking</code> (write — inserts a row, sends an email).
What it remembers	Persistent state lives in SQLite (bookings; shared with the web app; survives restarts). Within a conversation, Claude holds the working date / slot context. Availability is derived from bookings + rules, never separately stored.
What it must never do	Never double-book a slot (enforced by a partial unique index), never book on a holiday / closed day, never book without name + email, never reveal another player's booking or code.
Human-in-the-loop	Read tools run autonomously. The write action (<code>create_booking</code>) is confirmed with the user first, and Claude Desktop additionally prompts the user to approve each tool call before it runs.

Field	Detail
When something breaks	Slot already taken → the unique-index error is caught and returned as a friendly “that slot's gone” with alternatives. Empty / failed tool result → Claude reports it and asks rather than guessing. Email-send failure → logged, the booking still succeeds, and the user is told the invite didn't go out.
How I know it worked	End-to-end: a booking completes, an ARENA-XXXX code is returned, a row lands in SQLite, and that code unlocks the web mini-game. Target: about 9/10 successful bookings, each under 2 minutes.

3 Project Overview

VirtualSoccerArena (repo: **soccer-arena-mcp**) is an agentic booking system for a fictional soccer arena in Toronto, Ontario. The agent is Claude, running in Claude Desktop; the project provides the tools, state, and business logic the agent orchestrates through the Model Context Protocol (MCP).

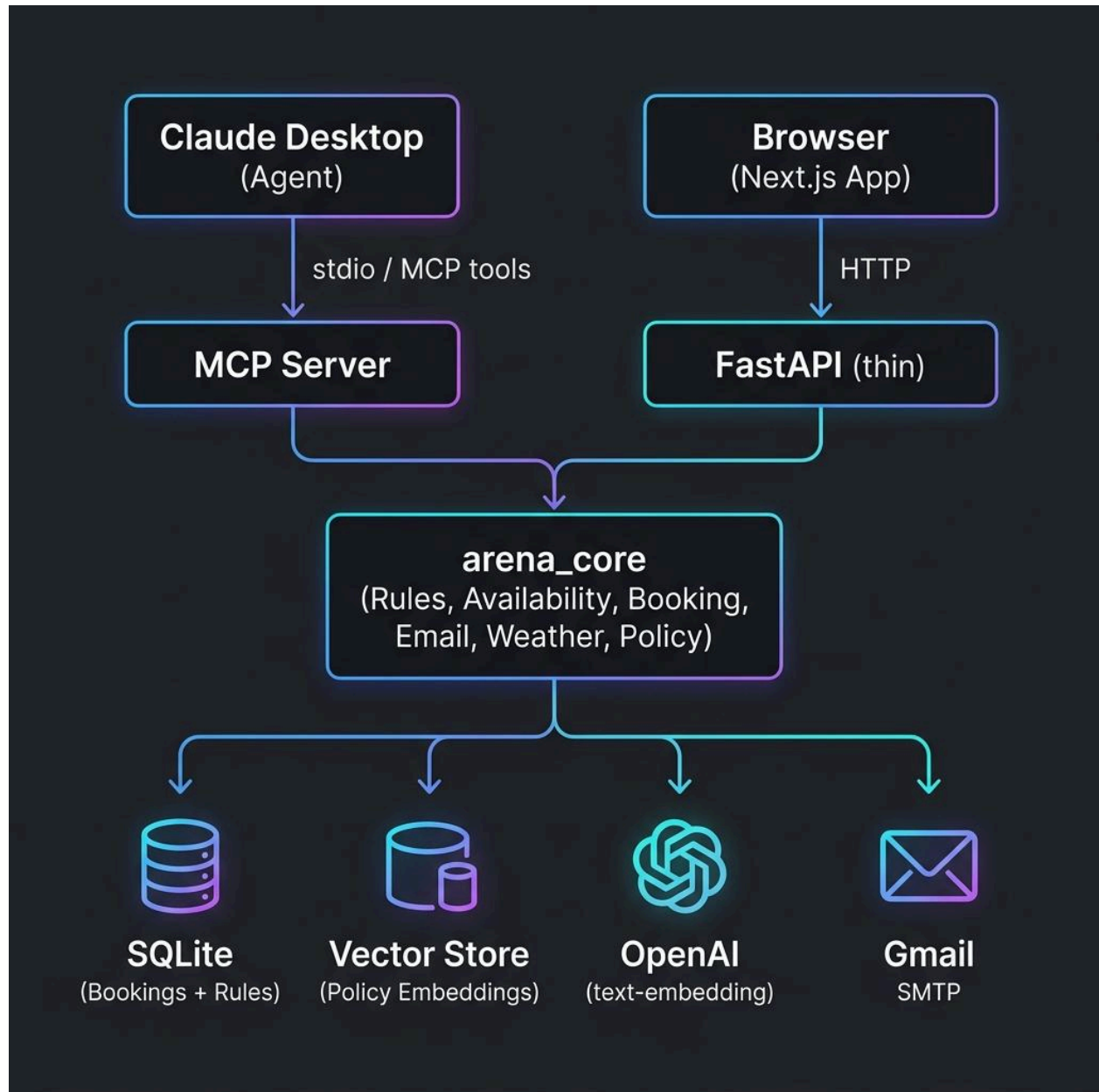
The headline workflow is genuinely multi-step and stateful — not a single LLM call. A user asks a loose question (“anything open Saturday evening?”), and Claude chains tool calls: checking availability, answering a policy question, confirming details, then writing a booking — recovering when a slot is taken, and pausing for human confirmation before the write. A booking made through Claude lands in the same SQLite store the Next.js web app reads, so the code Claude returns unlocks a custom cyberpunk robot-goalkeeper mini-game in the browser (behind a CN Tower skyline backdrop) — proving Claude and the web app share one source of truth.

Four features

1. **Policy check** — RAG over a ~100-page policy PDF; LlamaIndex retrieves the relevant passages and Claude synthesizes the answer.
2. **Availability & rates** — computed from rules (open hours, peak/off-peak, weekend uplift, Ontario holiday closures), minus already-booked slots.
3. **Booking & confirmation** — returns a unique code and emails an .ics calendar invite.
4. **Weather forecast** — mock seasonal data for the demo window (Jun–Dec 2026).

4 Architecture

All business logic lives in **arena_core** (Python). Two “front doors” import it: the MCP server (for Claude) and FastAPI (for the web app). Both open the same SQLite file, so a booking made by Claude is immediately visible to the web app. Availability is never stored — it's computed; bookings in SQLite are the single source of truth.



Component	Responsibility
arena_core (Python)	Single source of truth — business logic + data access.
MCP server (Python, stdio)	Wraps arena_core functions as the agent's tools; registered in Claude Desktop as virtualsoccerarena .
FastAPI (Python)	Thin HTTP layer so the web app reaches the same logic.
SQLite	Shared file; bookings are the source of truth (WAL enabled so MCP and FastAPI can read/write concurrently).

Component	Responsibility
Next.js + TypeScript + Tailwind	Info pages + the code-locked goalkeeper mini-game.

Data layer: SQLite (bookings + rules) · Vector store (policy embeddings) · OpenAI text-embedding · Gmail SMTP.

5 Datasets Used

Dataset	What it is	Role
Policy corpus	A ~100-page policy PDF assembled from distinctly-written sections (cancellation, rebooking, late arrival, no-show, refunds, weather, conduct, equipment, group bookings, membership, payment, liability, minors, facility rules, booking codes, FAQ).	Source for the <code>query_policy</code> RAG tool.
Rules (in code)	Operating hours (weekday 08:00–20:00 → 6 sessions; weekend 06:00–22:00 → 8 sessions), 2-hour slots, peak/off-peak split, CAD rates (\$120/\$160 weekday, \$150/\$200 weekend), and the 7 Ontario statutory-holiday closures across Jun–Dec 2026.	Drives availability + pricing.
Mock weather	Deterministic, seasonal forecast (temperature + condition) keyed on date, Jun–Dec 2026.	The <code>get_weather</code> tool.
Bookings (runtime)	Name, email, date, slot, rate, unique code (<code>ARENA-XXXX</code>), status — written to SQLite.	The agent's persistent state.

Embeddings & vector store. Policy text is embedded with OpenAI `text-embedding-3-small` (512-dim) and stored in Pinecone; the Pinecone index dimension is matched to the embedding model. `query_policy` retrieves the top passages and Claude synthesizes the answer. Re-ingestion is idempotent — the index is cleared before re-upserting.

6 The Hard Parts — Control Flow, State, Errors, Human-in-the-Loop

These are what the project is graded on, so they are the core of the build, not an afterthought.

- **Control flow:** Claude chooses the tool sequence from intent. A booking request typically fans out to `check_availability` (and often `query_policy` / `get_weather`) before converging on `create_booking`. The tools' descriptions and input schemas are the design surface that makes the agent chain them correctly.
- **State:** SQLite is the durable store, with WAL mode so two processes can use it safely. Availability is computed from bookings + rules, so there is nothing to keep “in sync.” The same store backs both the agent and the web app, which is what lets a Claude-made booking unlock the browser game.
- **Error handling:** double-booking is rejected at the database layer (a partial unique index on confirmed slots) and surfaced to the agent as a recoverable “slot taken”; closed/holiday days return a clear closed reason instead of an empty slot list; email failures are caught so the booking still succeeds; policy re-ingestion clears the index first to avoid duplicate vectors.
- **Human-in-the-loop:** the only write tool, `create_booking`, is confirmed with the user before execution, and Claude Desktop's per-tool approval prompt adds a second gate. Read tools run autonomously. This follows the handout's rule: reads can be autonomous; writes deserve a human.

7 Prompts Used During Vibe Coding

Representative prompts in the right shape — replace with the exact prompts you used in your AI coding tool.

1. *“I want to build a MCP for virualsoccerArena web app. This is a project I'm building as part of cohort learning. don't give me whole plan yet. let's discuss back and forth. finally I will ask to create a .Md with detailed implementation plan based on our discussion. wait until I say to create. this web app has 4 main features policy check — documents should have policies about cancelation, rebooking, late coming, etc etc details in pdf document availability and rate — rates are low on non peak hours vs peak hours. weekend rate are higher than weekdays. all national holidays of canada — arena is closed. weekdays there are 6 sessions available — 2 hours each (8am open – 8pm close) weekends 8 sessions available — 2 hours each (6am open – 10pm close) booking and confirmation — should return a unique code and calender invite with time and date weather forecast — temperature, sunny-rainy-overcast, date-june,July,Aug mock data this web app should have a doodle soccer play — user should be able to enable play button with unique code he got during booking confirmation. tech stack — I want this to be deployed on vercel — need modern UI not streamlit — policy documents I want to use llama index open libraries — mem0 in memory may be good for availability and dates — booking database I think is same as availability ? or how do we keep it in sync ? — LLM will be nemotron mini through nembus token factory. finally thought*

is, this MCP should be configured in claud to demo booking. now let's develop this idea. ask me questions and suggest any corrections / recommendations.”

Note: After prompt 1 — had back and forth conversation and polished the idea with many changes.

2. “Create implementation plan with corrections/changes” — Validated implementation plan.
3. “Go ahead with implementation plan”

8 Iterations I Tried

Real design iterations — what I explored, and the choice I shipped.

1. **Storage: mem0 → SQLite.** Considered an in-memory layer for availability/bookings, then realised availability is derived, not stored, and transactional state needs real persistence. Shipped SQLite.
2. **A second LLM (Nemotron) → none.** Planned a separate model for the app, then cut it — in an MCP architecture, Claude is the LLM consuming the tools, so a second model was redundant.
3. **In-memory single backend → SQLite (again).** Reconsidered one always-on in-memory backend; rejected it because the MCP server and the web app are separate processes, so in-process memory wouldn't be shared and would vanish on restart. A shared SQLite file solves both.
4. **Vector store: evaluated local Chroma vs Pinecone.** Weighed a fully-local Chroma store against a hosted one; shipped Pinecone for the RAG index.
5. **Rules: evaluated DB-driven vs in-code.** Designed a variant with hours/rates/holidays in SQLite tables; shipped the in-code rules for simplicity, keeping the data-driven version as a documented alternative.
6. **RAG / LlamaIndex: kept deliberately.** The policy corpus is small enough to fit in context, so RAG isn't strictly necessary — kept it as a learning objective, knowing it's intentional overkill for the data size.

9 Learnings & Observations

- **MCP servers contain no LLM.** The client (Claude) is the brain; the server is a menu of real, executable tools. Tool descriptions are a genuine design surface — vague ones make the agent call the wrong thing.
- **Derived state beats synced state.** Treating availability as a query over bookings + rules removed an entire class of “keep two things in sync” bugs.
- **In-memory doesn't cross process boundaries.** The single most useful realisation: two processes can only share state through something external (a file, a server) — not in-process memory. This is why SQLite, not an in-memory store, backs the demo.
- **Right-size the RAG.** A vector pipeline is justified by corpus size, not by reflex. For a small policy set, retrieval is optional; the discipline is knowing why you're using it.

- **Write actions need a gate.** Booking creates a record and sends an email — exactly the kind of action that should pause for human confirmation. Claude Desktop's tool-approval prompt makes human-in-the-loop almost free.
- **Practical gotchas:** Gmail needs an App Password (2FA on), not the login password; the Pinecone index dimension must equal the embedding model's output (512 for `text-embedding-3-small`); SQLite needs WAL for two-process access.

10 How I Know It Works (Success Criteria & Tests)

- A natural request (“book Saturday 6pm for Alex, alex@example.com”) yields a confirmed booking, a returned `ARENA-XXXX` code, and a row in SQLite.
- The same code unlocks the goalkeeper mini-game — proving the shared store end-to-end.
- A second booking attempt on a taken slot is refused gracefully, not crashed.
- A holiday date returns closed, not an empty/garbage slot list.

Automated tests (pytest) cover: day-type detection, sessions per day type, peak/off-peak classification, the rate table, holiday closures, availability subtracting confirmed bookings, double-booking rejection, and the booking-code alphabet being collision-safe. A `docs/demo-prompts.md` file scripts 10 prompts that exercise every tool, edge case, and the end-to-end unlock flow.